

MPI & OPENMP

天主教輔仁大學-資訊工程所

Multi-core programming

Oct 24th, 2020

周賜福

Department of CSIE

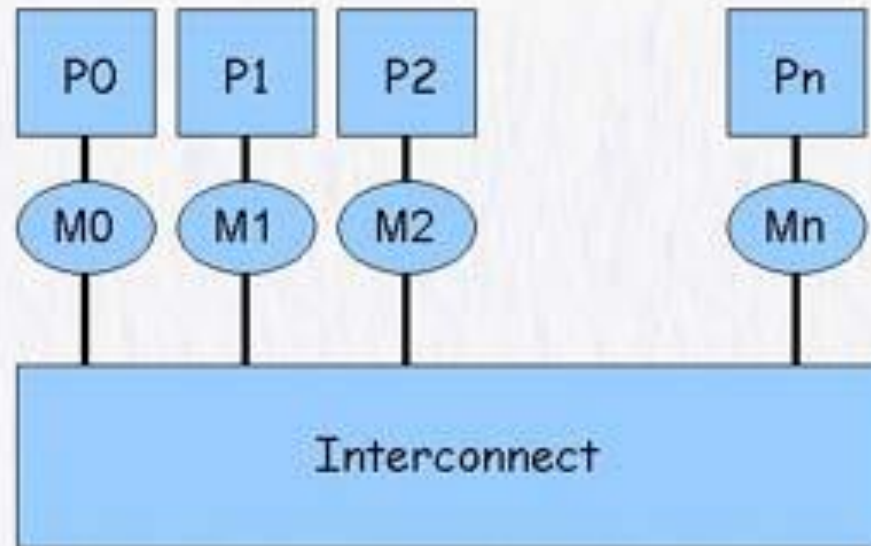
Outline

- Message-passing model
- Message Passing Interface (MPI)
- Coding MPI programs
- What is OPENMP?
- How to parallelize the loop?
- What is task level parallelism?
- Example programs.

Different types of parallel platforms:

Distributed Memory

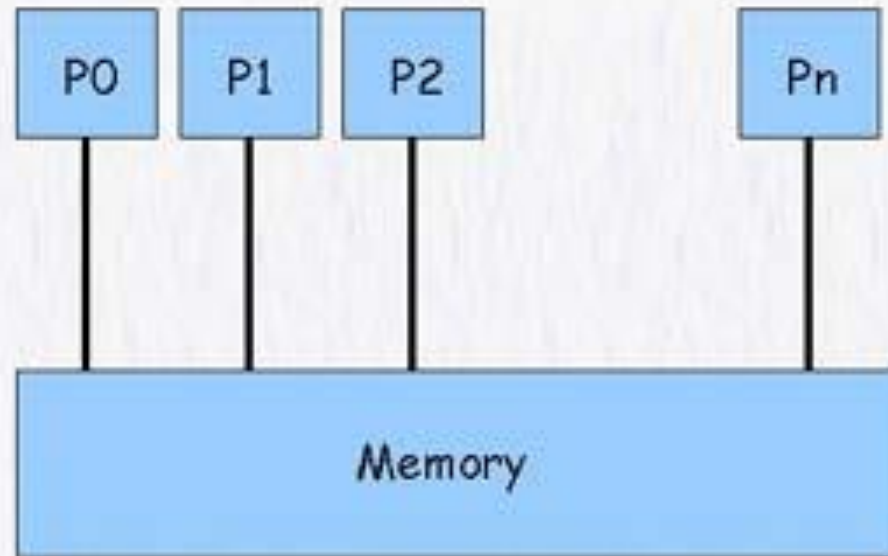
M0, M1, ... Mn are memories associated with processors P0, P1, ..., Pn.



Distributed Memory Architecture

Different types of parallel platforms: Shared Memory

$P_0, P_1, \dots, P_n$ are processors.



Shared Memory Architecture

Different types of parallel platforms:

Shared Memory

- SMP: Symmetric Multiprocessing
 - Identical processing units working from the same main memory
 - SMP machines are becoming more common in the everyday workplace
 - Dual-socket motherboards are very common, and quad-sockets are not uncommon
 - 2 and 4 core CPUs are now commonplace
 - Intel Larabee: 12-48 cores in 2009-2010
- ASMP: Asymmetric Multiprocessing
 - Not all processing units are identical
 - One processor from each floor can be connected together.

Parallel Programming Models

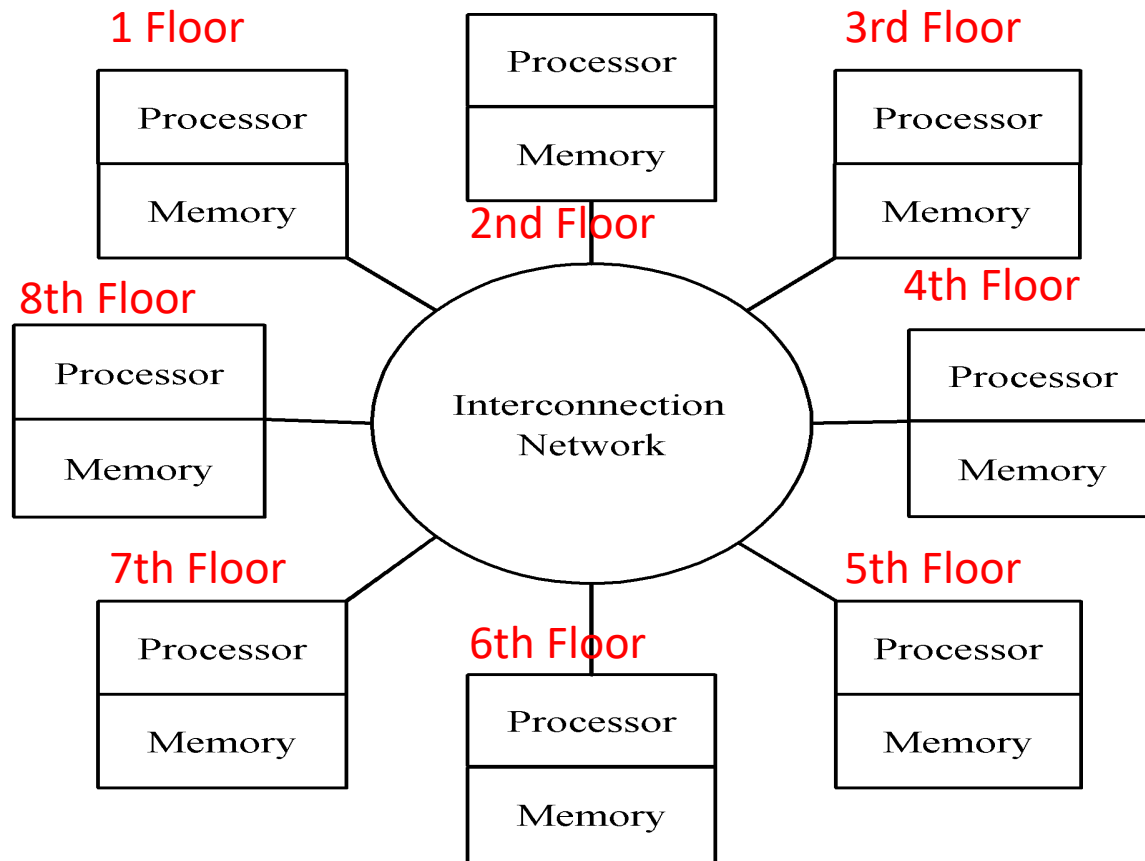
- Shared Memory
 - Multiple processors sharing the same memory space
- Message Passing
 - Users make calls that explicitly share information between execution entities
- Remote Memory Access
 - Processors can directly access memory on another processor
- These models are then used to build more sophisticated models
 - Loop Driven
 - Function Driven Parallel (Task-Level)

Shared Memory Programming

- SysV memory manipulation
 - One can actually create, manipulate, shared memory spaces.
- Pthreads (Posix Threads)
 - Lower level Unix library to build multi-threaded programs
- OpenMP (www.openmp.org)
 - Protocol designed to provide automatic parallelization through compiler pragmas.
 - Mainly loop driven parallelism
 - Best suited to desktop and small SMP computers
- Caution: Race Conditions
 - When two threads are changing the same memory location at the same time.

MPI – Message Passing Interface

Message-passing Model



Task/Channel vs. Message-passing

Task/Channel	Message-passing
Task	Process
Explicit channels	Any-to-any communication

Processes for the MPI Model

- Number is specified at start-up time
- Remains constant throughout execution of program
- All execute the same program
- Each has unique ID number
- Alternately performs computations and communicates

Advantages of Message-passing Model

- Gives programmer ability to manage the memory hierarchy
- Portability to many architectures
- Easier to create a deterministic program
- Simplifies debugging

The Message Passing Interface

- Late 1980s: vendors had unique libraries
- 1989: Parallel Virtual Machine (PVM) developed at Oak Ridge National Lab
- 1992: Work on MPI standard begun
- 1994: Version 1.0 of MPI standard
- 1997: Version 2.0 of MPI standard
- Today: MPI is dominant message passing library standard

First MPI program hello world

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>                /* MPI header file */
int main (int argc, char *argv[])
{
    /* Process rank */
    int nprocs;
    int myid;
    /* initialize for MPI (should come before any other calls to MPI routines) */
    if (MPI_Init(&argc, &argv) != MPI_SUCCESS) {
        fprintf(stderr, "MPI initialization error\n"); exit(EXIT_FAILURE);
    }
    /* get number of processes (function returns success/error, but we ignore it) */
    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);
    /* print a greeting */
    printf("hello from process %d of %d\n", myid, nprocs);
    fflush (stdout);
    /* clean up for MPI (should come after all other calls to MPI routines)
    (function returns success/error, but we ignore it) */
    MPI_Finalize();
    return EXIT_SUCCESS;
}
```

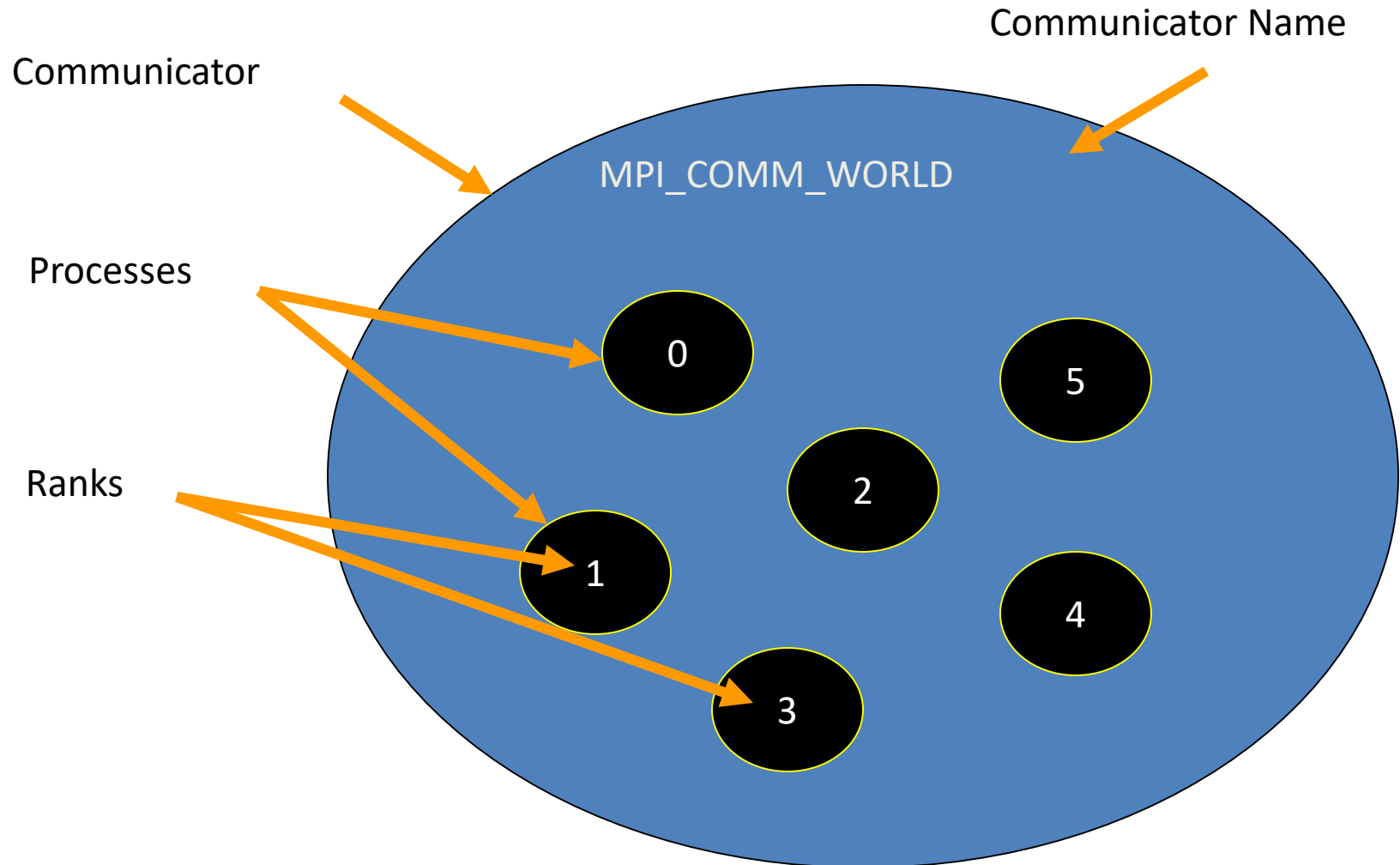
How to Compile and run MPI program?

Store the above program as hello.c

Start MPI environment using the following command.

```
%  
%      mpicc -o hello hello.c  
%      hello executable file will be created.  
%      mpirun -np 2 hello  
%      hello from process 0 of 2  
      hello from process 1 of 2  
%      mpirun -np 4 hello  
      hello from process 0 of 4  
      hello from process 1 of 4  
      hello from process 2 of 4  
      hello from process 3 of 4
```

Communicator

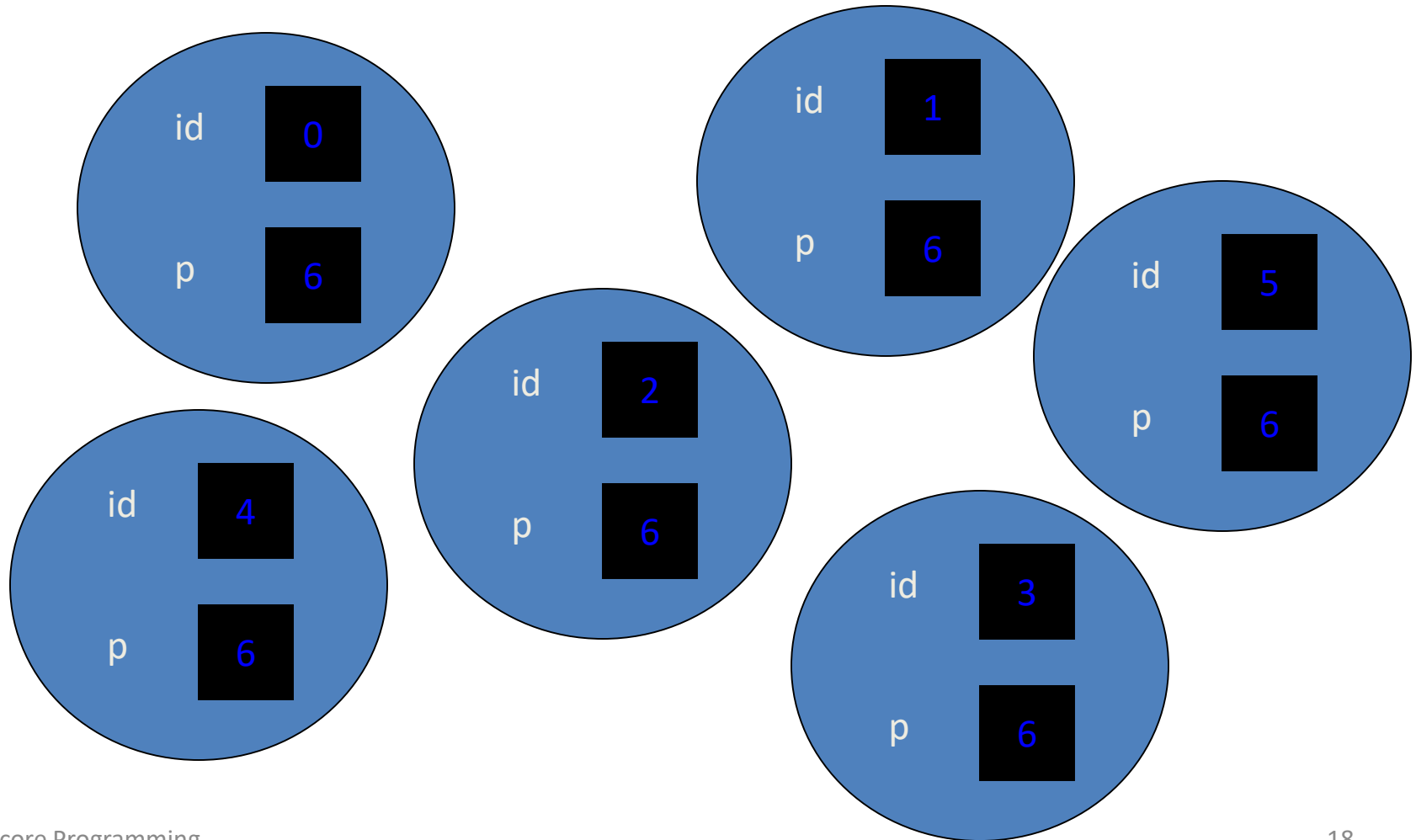


Determine Number of Processes

```
MPI_Comm_size (MPI_COMM_WORLD, &p) ;
```

- First argument is communicator
- Number of processes returned through second argument

Replication of Automatic Variables



We will not work on MPI because, we need to
focus on OPENMP

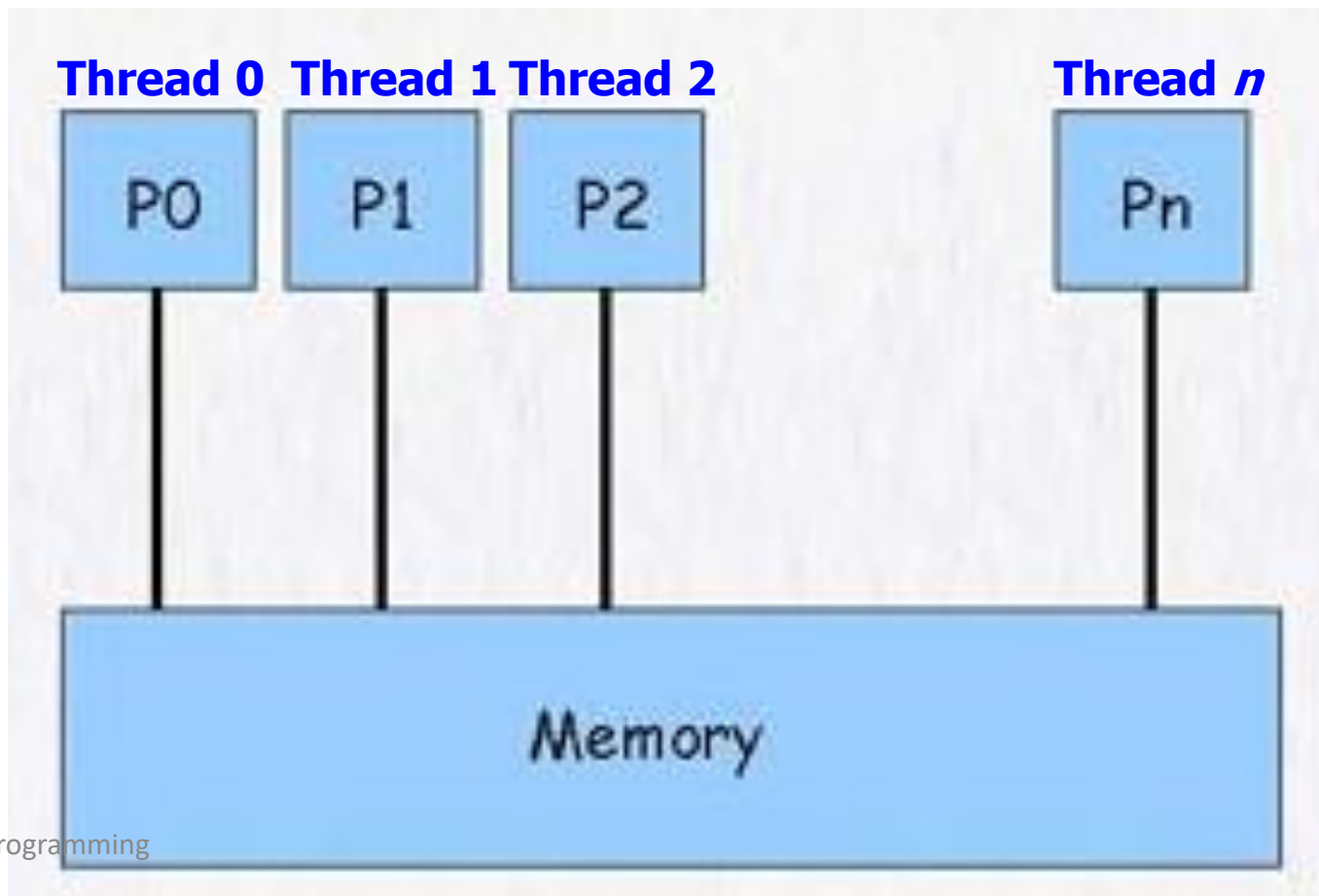
OPENMP Programming

Introduction

- OpenMP is designed for shared memory systems.
- OpenMP is easy to use
 - achieve parallelism through compiler directives
 - or the occasional function call
- OpenMP is a “quick and dirty” way of parallelizing a program.
- OpenMP is usually used on existing serial programs to achieve moderate parallelism with relatively little effort

Computational Threads

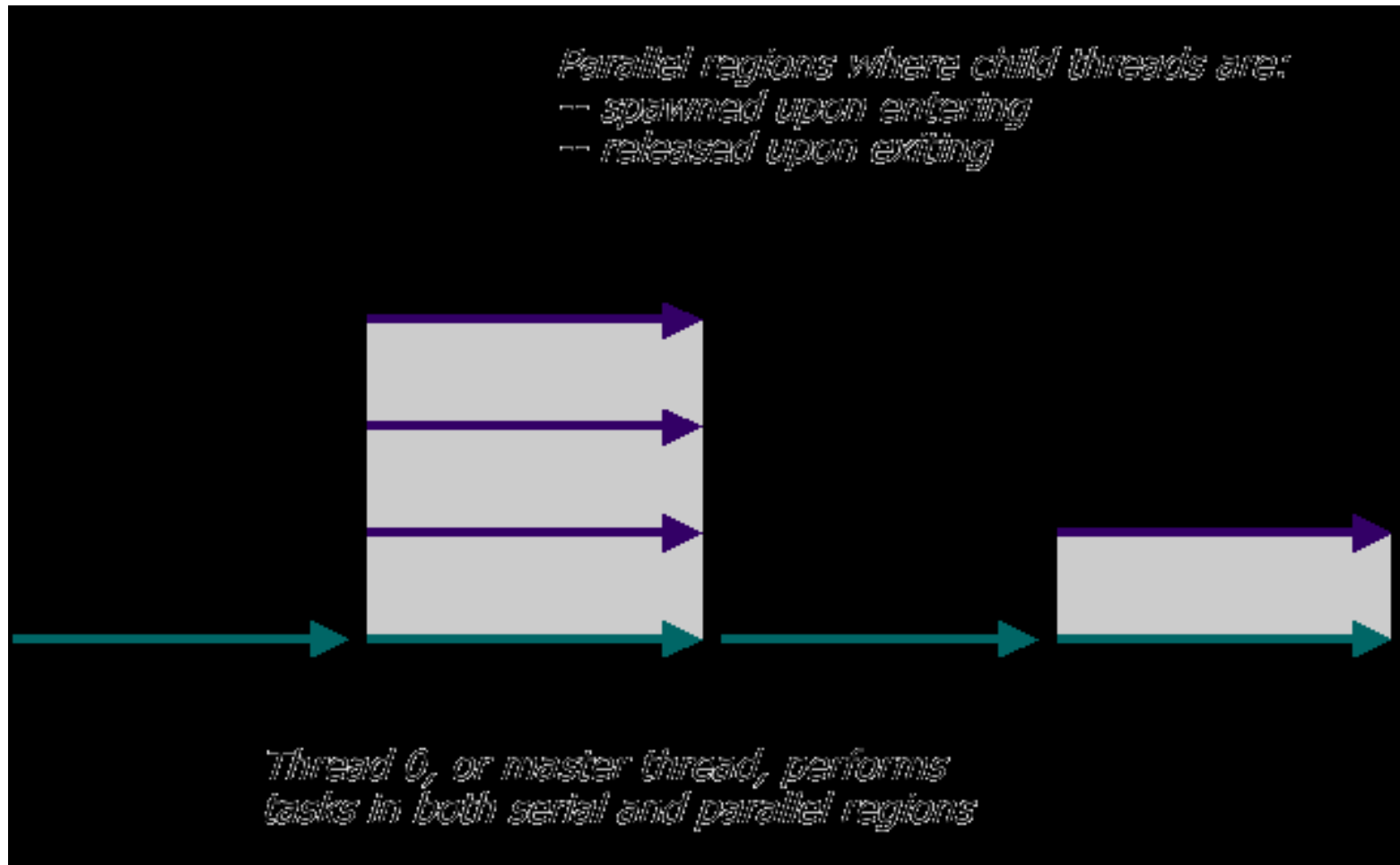
- Each processor has one thread assigned to it
- Each thread runs one copy of your program



OpenMP Execution Model

- In MPI, all threads are active all the time
- In OpenMP, execution begins only on the master thread. Child threads are spawned and released as needed.
 - Threads are spawned when program enters a **parallel region**.
 - Threads are released when program exits a **parallel region**

OpenMP Execution Model



Parallel Region Example: For loop

Fortran:

```
!$omp parallel do
do i = 1, n
    a(i) = b(i) + c(i)
enddo
```

This comment or pragma tells openmp compiler to spawn threads *and* distribute work among those threads

These actions are combined here but they can be specified separately between the threads

C/C++:

```
#pragma omp parallel for
for(i=1; i<=n; i++)
    a[i] = b[i] + c[i];
```

Pros of OpenMP

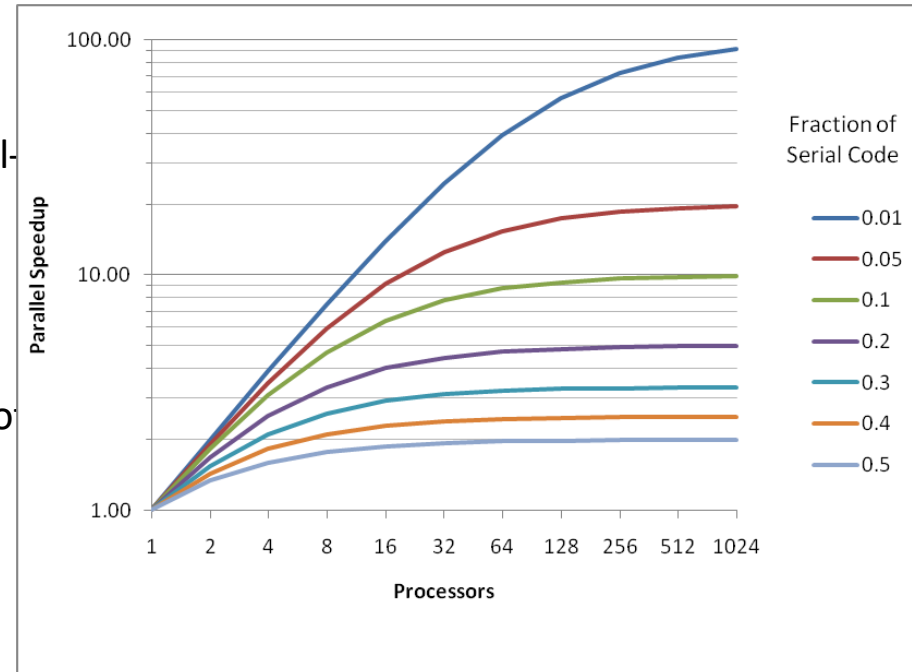
- Because it takes advantage of shared memory, the programmer does not need to worry (that much) about data placement
- Programming model is “serial-like” and thus conceptually simpler than message passing
- Compiler directives are generally simple and easy to use
- Legacy serial code does not need to be rewritten

Cons of OpenMP

- Codes can only be run in shared memory environments!
 - In general, shared memory machines beyond ~8 CPUs are much more expensive than distributed memory ones, so finding a shared memory system to run on may be difficult
- Compiler must support OpenMP
 - whereas MPI can be installed anywhere
 - However, gcc 4.2 now supports OpenMP

Cons of OpenMP

- In general, only moderate speedups can be achieved.
 - Because OpenMP codes tend to have serial-only portions, Amdahl's Law prohibits substantial speedups
- Amdahl's Law:
 F = Fraction of serial execution time that cannot be parallelized
 N = Number of processors



$$\text{Execution time} = \frac{1}{F + (1 - F)/N}$$

If you have big loops that dominate execution time, these are ideal targets for OpenMP

Goals of this lecture

- Exposure to OpenMP
 - Understand where OpenMP may be useful to you now
 - Or perhaps 4 years from now when you need to parallelize a serial program, you will say, “Hey! I can use OpenMP.”
- Avoidance of common pitfalls
 - How to make your OpenMP actually get the same answer that it did in serial
 - A few tips on dramatically increasing the performance of OpenMP applications

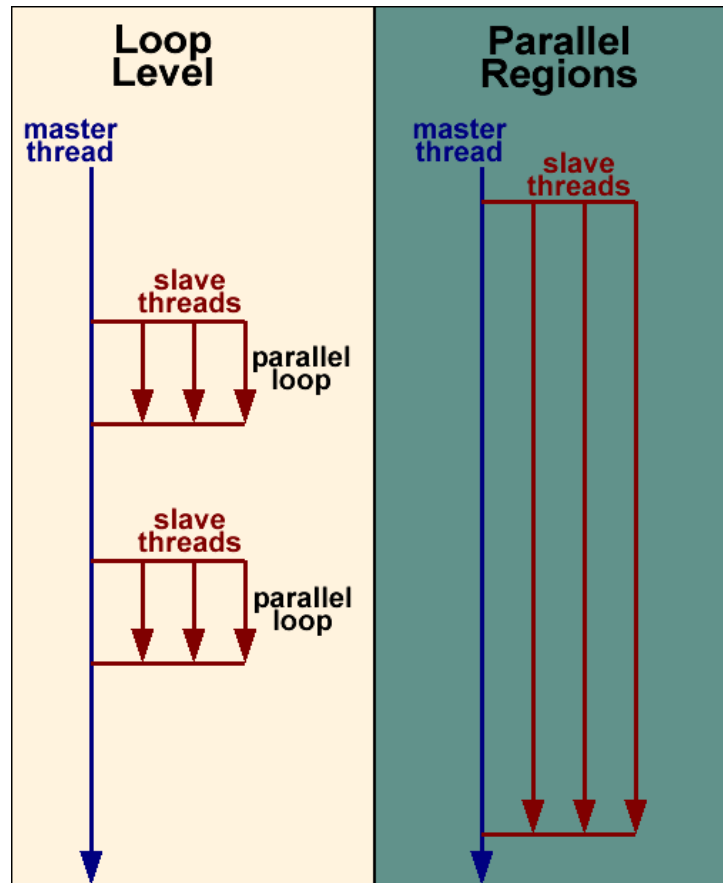
Compiling and Running OpenMP

- True64: -mp
- SGI IRIX: -mp
- IBM AIX: -qsmp=omp
- Portland Group: -mp
- Intel: -openmp
- gcc (4.2) -fopenmp

OpenMP Basics:

2 Approaches to Parallelism

Divide loop iterations among threads: We will focus mainly on loop level parallelism in this lecture



Divide various sections of code between threads

Sections: Functional parallelism

```
#pragma omp parallel
{
    #pragma omp sections
    {
        #pragma omp section
        block1
        #pragma omp section
        block2
    }
}
```

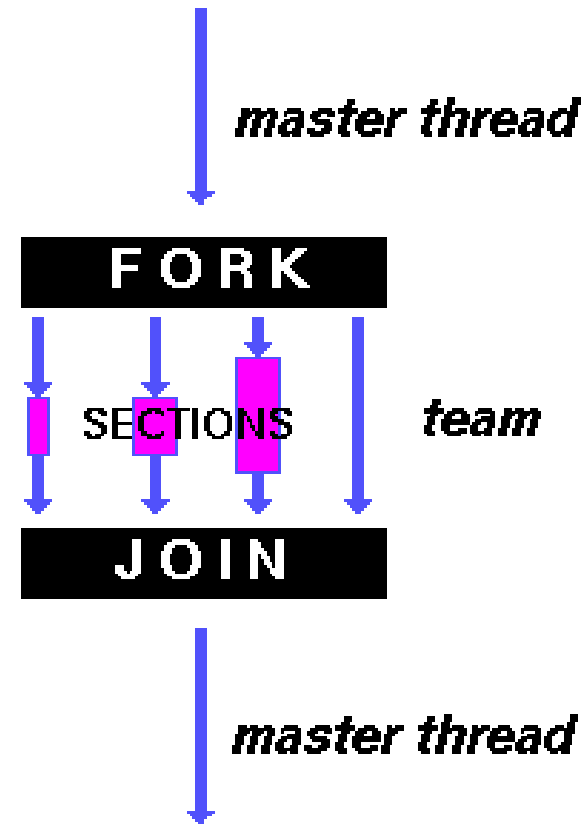


Image from:
<https://computing.llnl.gov/tutorials/openMP>

Parallel DO/for: Loop level parallelism

Fortran:

```
!$omp parallel do
do i = 1, n
    a(i) = b(i) + c(i)
enddo
```

C/C++:

```
#pragma omp parallel for
for(i=1; i<=n; i++)
    a[i] = b[i] + c[i];
```

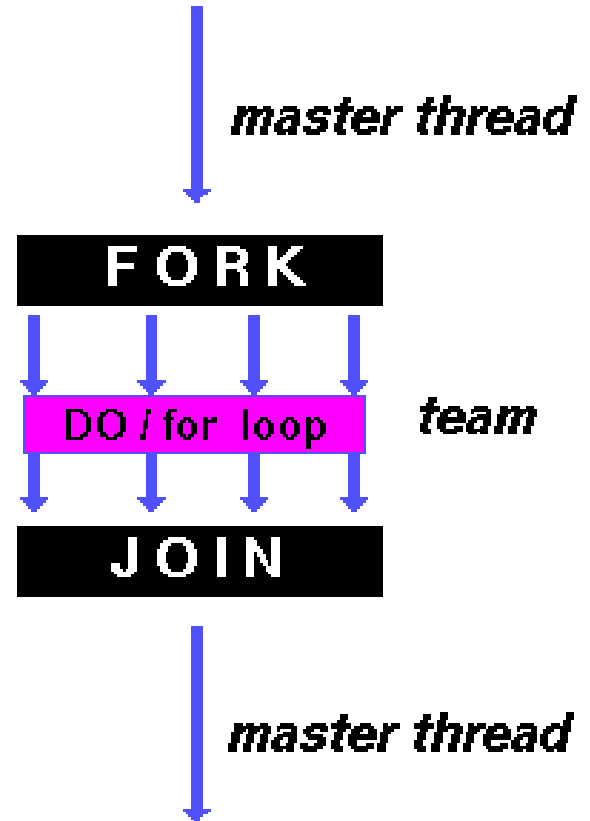


Image from:
<https://computing.llnl.gov/tutorials/openMP>

Pitfall #1: Data dependencies

- Consider the following code:

```
a[0] = 1;  
for(i=1; i<5; i++)  
    a[i] = i + a[i-1];
```

- There are dependencies between loop iterations.
- Sections of loops split between threads will not necessarily execute in order
- Out of order loop execution will result in undefined behavior

Pitfall #1: Data dependencies

- 3 simple rules for data dependencies
 1. All assignments are performed on arrays.
 2. Each element of an array is assigned to by at most one iteration.
 3. No loop iteration reads array elements modified by any other iteration.

How can we convert the C program into
parallel using OpenMP

Simple Hello program helloomp.c

```
#include <stdio.h>
```

```
int main() {
```

```
    int i;
```

```
    printf("Hello World\n");
```

```
    for(i=0;i<3;i++)
```

```
        printf("Printing Hello :%d\n",i);
```

```
    printf("GoodBye World\n");
```

```
}
```

Compile the program using gcc and observe

```
gcc -o helloomp helloomp.c
```

The Output you get is as follows:

Hello World

Iter:0

Iter:1

Iter:2

GoodBye World

Let us no add the OMP parallelization.

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
int main() {
```

```
    #pragma omp parallel
```

```
{
```

```
    int i;
```

```
    printf("Hello World\n");
```

```
    for(i=0;i<3;i++)
```

```
}
```

```
    printf("Printing Hello :%d\n",i);
```

```
    printf("GoodBye World\n");
```

```
}
```

Compile the program using **-fopenmp** and observe

`gcc -fopenmp -o helloomp helloomp.c`

Hello World	Hello World	
Hello World	Iter:0	Iter:2
Iter:0	Iter:1	Iter:2
Iter:1	Iter:2	Hello World
Iter:2	Iter:0	Iter:0
Hello World	Iter:1	Iter:1
Iter:0	Iter:2	Iter:2
Iter:1	Hello World	GoodBye World
Hello World	Iter:0	
Hello World	Iter:1	
Iter:0	Iter:2	
Iter:1	Iter:0	
Iter:2	Iter:1	

Generic development Cycle

Windows thread

main.cpp

```
#include <stdio.h>
#include <windows.h>
const int numThreads = 4;
DWORD WINAPI helloFunc(LPVOID pArg)
{
    printf("Hello Thread\n");
    return 0;
}
```

See how it is
run under
openMP compiler?

```
int main()
{
    HANDLE hThread[numThreads];

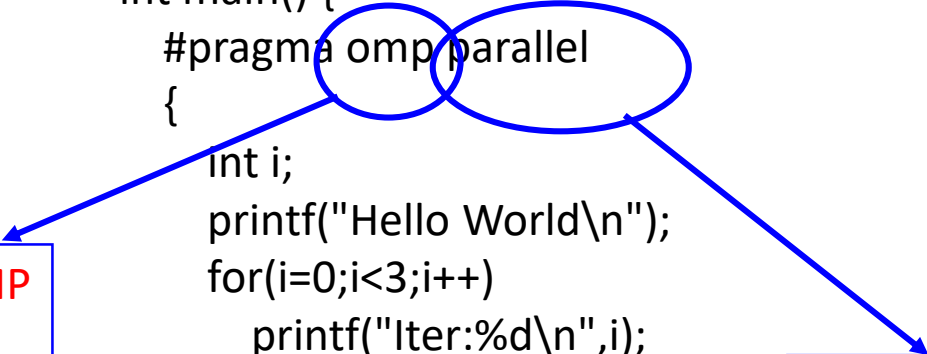
    for (int i = 0; i < numThreads; i++)
        hThread[i] =
            CreateThread(NULL, 0, helloFunc, NULL, 0, NULL );

    WaitForMultipleObjects(numThreads, hThread, TRUE, INFINITE);
    return 0;
}
```

Using OpenMP create multiple threads

helloomp.c

```
#include <stdio.h>
#include <omp.h>
int main() {
    #pragma omp parallel
    {
        int i;
        printf("Hello World\n");
        for(i=0;i<3;i++)
            printf("Iter:%d\n",i);
    }
    printf("GoodBye World\n");
}
```



openMP
Region

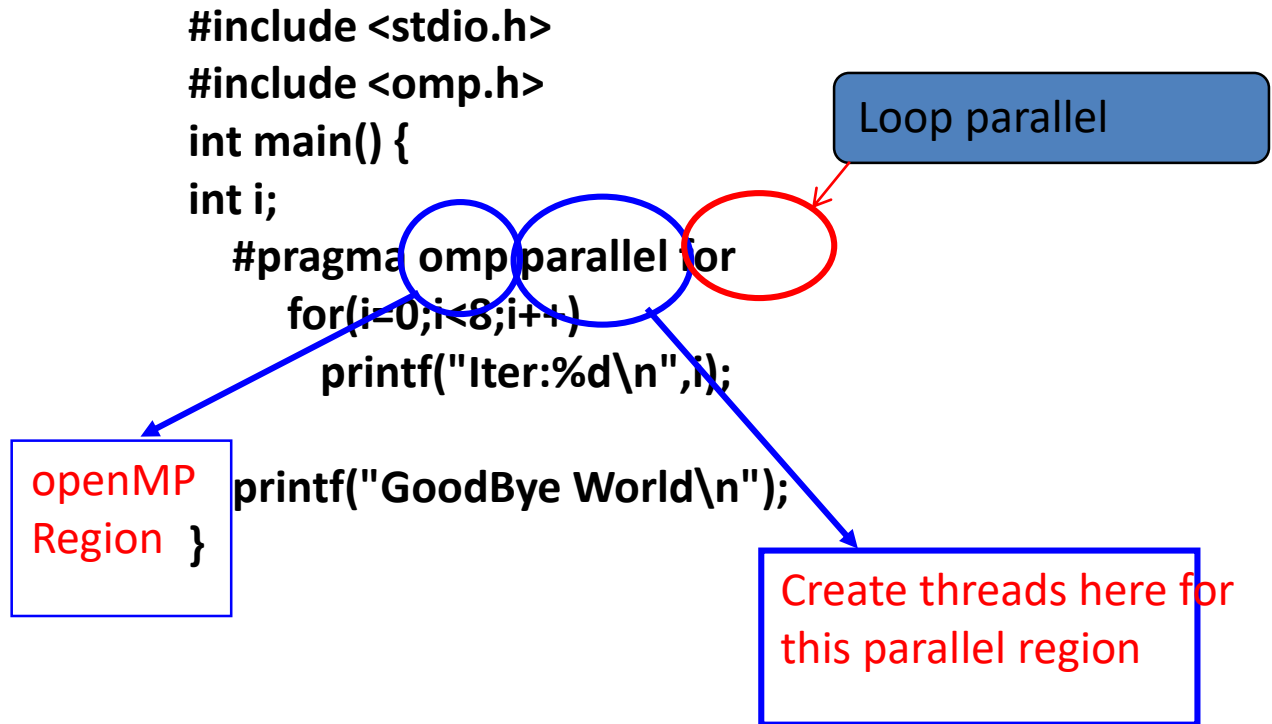
**Run the program and
observe**

What is the difference
between OpenMP
thread created here
and the windows thread?

Create threads here for
this parallel region

Using OpenMP create multiple threads

helloomp1.c



Compiling openMP in UNIX environment

```
gcc -fopenmp -o helloomp1 helloomp1.c
```

The output of the program is as follows:

Iter:5

Iter:7

Iter:4

Iter:0

Iter:3

Iter:2

Iter:1

Iter:6

GoodBye World

Another way of writing using Parallel section

```
#include<stdio.h>
int main( int ac, char **av)
{
    #pragma omp parallel
    // specify the code between the curly brackets is part of an
    //OpenMP parallel section.
    {
        printf("Hello World!!!\n");
    }
    return 0;
}
```

Result of running the program on the previous slide.

Hello World!!!
Hello World!!!
Hello World!!!
Hello World!!!
Hello World!!!
Hello World!!!
Hello World!!!
Hello World!!!

Another Example to understand
what is critical section

Let us take a program that runs for a long time and see how we can use thread to reduce the time.

Program to calculate pi (pi.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(){
    clock_t start, stop;
    unsigned int iter=2000000000;
    int i,j;
    double x, y;
    double dUnderCurve=0.0;
    double pi=0.0;
    start=clock();
    srand( 0);
    for (i=0;i<iter;i++) {
        x=(double)rand()/(double)RAND_MAX;    //X Coordinate
        y=(double)rand()/(double)RAND_MAX;    //Y Coordinate
        if (x*x + y*y <= 1.0) { //is distance from Origin under Curve
            dUnderCurve++;
        }
    }
    pi = dUnderCurve / (double) iter * 4 ;
    stop=clock();

    printf ("pi = %10.9f\n", pi);
    printf ("Seconds = %10.9f\n", (double)(stop-start)/ CLOCKS_PER_SEC);
    return 0;
}
```

It took roughly about =
37.010016000 Seconds

Pi must be 3.14159..

Can we improve the serial version and see how we can speedup

Pi calculation without Y co-ordinate

```
#include <stdio.h>
```

```
#include <time.h>
```

Pi-serial.c

```
long long num_steps = 2000000000;  
double step;
```

**Run this program and observe the
time it took. (2000 000 00)**

```
int main(int argc, char* argv[])  
{
```

```
    clock_t start, stop;
```

```
    double x, pi, sum=0.0;
```

```
    int i;
```

```
    step = 1./((double)num_steps);
```

```
    start = clock();
```

```
    for (i=0; i<num_steps; i++)
```

```
    {
```

```
        x = (i + .5)*step;
```

```
        sum = sum + 4.0/(1.+ x*x);
```

```
    }
```

```
    pi = sum*step;
```

```
    stop = clock();
```

```
    printf("The value of PI is %15.12f\n",pi);
```

```
    printf("The time to calculate PI was %f seconds\n",((double)(stop - start)/ CLOCKS_PER_SEC ));
```

```
    return 0;
```

```
}
```

**It took roughly about
13.919599 seconds**

Let us parallelize using
openmp

Pi-program using openMp parallel for loop

piomppara.c

```
#include <stdio.h>
```

```
#include <time.h>
```

```
long long num_steps = 2000000000;
```

```
double step;
```

```
int main(int argc, char* argv[])
```

```
{
```

```
    clock_t start, stop;
```

```
    double x, pi, sum=0.0;
```

```
    int i;
```

```
    step = 1./((double)num_steps);
```

```
    start = clock();
```

```
    #pragma omp parallel for
```

```
    for (i=0; i<num_steps; i++)
```

```
    {
```

```
        x = (i + .5)*step;
```

```
        sum = sum + 4.0/(1. + x*x);
```

openMP
Region

```
    pi = sum*step;
```

```
    stop = clock();
```

```
    printf("The value of PI is %15.12f\n", pi);
```

```
    printf("The time to calculate PI was %f seconds\n", ((double)(stop - start)/CLOCK_PER_SEC));
```

```
    return 0;
```

```
}
```

Added this to make it parallel

Observe what happens?

What is the time after
running this program?

The value of PI is 0.14590436003

The time to calculate PI was

87.524445seconds

Using thread 27 is reduced to 14

Let us continue Next week and analyze this program and see what happens?